

Unsolvable vs. Intractable Problems, P vs. NP, and NP-Completeness

Turing Machines

What is a Turing Machine?

A **Turing Machine** (TM) is an abstract mathematical model of computation introduced by Alan Turing in 1936. It is used to formally define what it means for a function or problem to be *computable*.

Key Idea: A Turing Machine is like a very simple computer:

- It has a *tape* divided into cells, each cell holding one symbol.
- It has a *head* that can read from and write to the tape.
- It follows a set of *rules* (the program) that tell it:
 1. What symbol to write,
 2. How to move (left or right),
 3. Which state to go to next.

How it Works (Step-by-Step)

1. The head reads the current tape symbol.
2. Based on the current state and the read symbol, the TM:
 - Writes a symbol (possibly replacing the old one),
 - Moves the head left or right,
 - Changes to a new state.
3. This process repeats until the machine enters the accept or reject state.

Why Turing Machines Matter

- They provide a precise definition of an *algorithm*.
- They can simulate any modern computer program (Church–Turing thesis).
- They help distinguish between **solvable** and **unsolvable** problems.

Example: Palindrome Checker (Conceptual)

A TM can be designed to check if a given string over $\{a, b\}$ is a palindrome:

1. Mark the leftmost symbol, find and mark the matching rightmost symbol.
2. Repeat for the remaining substring.
3. Accept if all symbols match; reject otherwise.

Remember: A Turing Machine is not a real physical machine—it's a **thought experiment** to study the limits of computation.

1. Unsolvable Problems

Definition

A problem is **unsolvable** (or *undecidable*) if there is no algorithm that can produce a correct answer for every valid input in a finite amount of time.

Example: The Halting Problem

Given a description of a program P and an input x , determine whether P halts on x . Alan Turing proved in 1936 that no algorithm can solve this problem for all possible P and x .

Key Idea

Unsolvable $\Rightarrow$ *impossible* to compute, no matter how powerful your computer is.

2. Intractable Problems

Definition

A problem is **intractable** if it can be solved in principle (there exists an algorithm), but all known algorithms require *unreasonable* amounts of resources (e.g., exponential time) for large inputs.

Example: Traveling Salesman Problem (TSP)

Given n cities and distances between them, find the shortest route visiting all cities exactly once and returning to the start. A brute-force approach requires $(n - 1)!$ checks, which grows extremely fast.

Key Idea

Intractable $\Rightarrow$ solvable in theory, but not feasible in practice for large instances.

3. P vs. NP

P (Polynomial Time)

Class **P** = set of decision problems that can be solved in polynomial time by a deterministic Turing machine.

Example: Checking if a number is prime (modern algorithms do this in polynomial time).

NP (Nondeterministic Polynomial Time)

Class **NP** = set of decision problems whose solutions, if given, can be *verified* in polynomial time.

Example: Given a specific route in TSP, you can quickly compute its length and check if it is below a given bound.

The Big Question

$$P \stackrel{?}{=} NP$$

Open problem: Does every problem whose solution can be verified quickly also have a quick method to find that solution? This is one of the seven Millennium Prize Problems.

4. NP-Completeness

Definition

A decision problem X is **NP-complete** if:

1. $X \in NP$ (solutions can be verified in polynomial time), and
2. Every problem in NP can be reduced to X in polynomial time. This means that for any problem Y in NP, there is a *polynomial-time transformation* (also called a **reduction**) that converts any instance of Y into an equivalent instance of X .

In other words:

- If you can solve X quickly, you can solve Y quickly by first transforming Y into X , then solving X .
- “Equivalent” means that the answer (*yes* or *no*) to the transformed instance of X will be the same as the answer to the original instance of Y .
- “Polynomial time” means that the transformation process itself takes at most $O(n^k)$ steps for some constant k , where n is the size of the input.

This requirement is what makes NP-complete problems the *hardest* in NP: if you can find a polynomial-time algorithm for one NP-complete problem X , then, via reductions, *every* NP problem can be solved in polynomial time.

Intuition

NP-complete problems are the *hardest* problems in NP. If you can find a polynomial-time algorithm for one NP-complete problem, you can solve all NP problems in polynomial time.

Examples

Here is a list of commonly studied NP-complete problems. These are decision problems, meaning the answer is always *Yes* or *No*.

1. Boolean Satisfiability Problem (SAT)
2. 3-SAT (SAT with 3 literals per clause)
3. Clique Problem
4. Independent Set Problem
5. Vertex Cover Problem
6. Hamiltonian Cycle Problem
7. Hamiltonian Path Problem
8. Traveling Salesman Problem (decision version)
9. Graph Coloring (for $k \geq 3$)
10. Subset Sum Problem
11. Knapsack Problem (decision version)
12. Partition Problem
13. Job Scheduling with Deadlines
14. Set Cover Problem
15. Exact Cover Problem
16. Dominating Set Problem
17. Feedback Vertex Set Problem
18. Longest Path Problem (decision version)
19. 0-1 Integer Programming
20. Steiner Tree Problem (decision version)
21. Directed Hamiltonian Cycle Problem

Note: For each problem, if you can find a polynomial-time algorithm for one NP-complete problem, all NP problems can also be solved in polynomial time through reductions.

9. Cook–Levin Theorem and the SAT Problem

Boolean Satisfiability Problem (SAT)

The **SAT problem** is the first problem that was proven to be NP-complete.

- **Input:** A Boolean formula, composed of variables, AND ($\wedge$), OR ($\vee$), and NOT ($\neg$) operators.
- **Question:** Is there an assignment of truth values (True/False) to the variables that makes the entire formula evaluate to True?

Example:

$$(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3)$$

Is there an assignment of $x_1, x_2, x_3 \in \{\text{True}, \text{False}\}$ that makes this formula True? Yes: $x_1 = \text{True}, x_2 = \text{False}, x_3 = \text{True}$.

Cook–Levin Theorem

- Proven independently by Stephen Cook (1971) and Leonid Levin (1973).
- **Statement:** SAT is NP-complete. That is:
 1. SAT $\in$ NP (given an assignment, we can check in polynomial time if it satisfies the formula).
 2. Every problem in NP can be *polynomial-time reduced* to SAT.
- **Significance:** This was the first proof that an NP-complete problem exists. It laid the foundation for the theory of NP-completeness.

Key Takeaway: SAT is the “gateway” NP-complete problem: showing a problem is as hard as SAT is the usual way to prove NP-completeness for other problems.

5. Summary Table

Category	Meaning
Unsolvable	No algorithm exists for all inputs (e.g., Halting Problem).
Intractable	Algorithm exists, but requires impractical resources for large inputs (e.g., TSP by brute force).
P	Solvable in polynomial time.
NP	Solutions can be verified in polynomial time.
NP-complete	Hardest problems in NP; solving one quickly solves them all.